Lecture 3: Chomsky Hierarchy, Decidability, and Computability

Formal Grammar Definition

A formal grammar of this type consists of a finite set of **production rules P** (left-hand side → right-hand side), where each side consists of a finite sequence of the following symbols:

- a finite set of **terminal symbols T** (indicating that no production rule can be applied)
- a finite set of **non-terminal symbols N** (indicating that some production rule can yet be applied)
- a **start symbol S** (a distinguished non-terminal symbol)

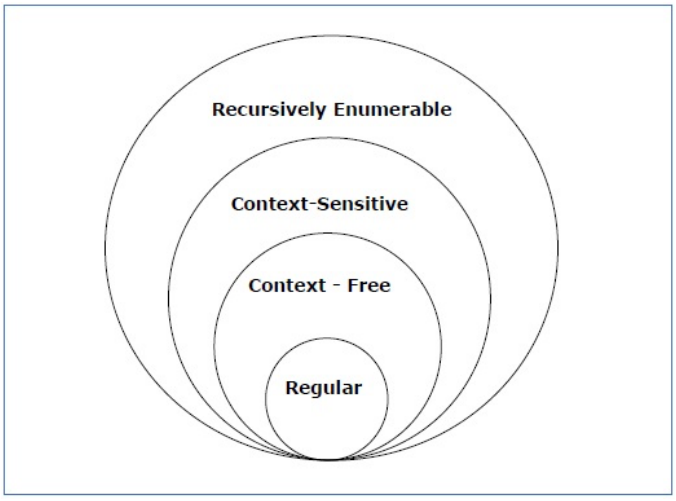
⇨ **G = (T, N, P, S)**

Chomsky Classification of Grammars

According to Noam Chomsky, there are four types of grammars − Type 0, Type 1, Type 2, and Type 3. The following table shows how they differ from each other.

Grammar Type	Grammar Accepted	Language Accepted
Type 0	Unrestricted grammar	Recursively enumerable language
Type 1	Context-sensitive grammar	Context-sensitive language
Type 2	Context-free grammar	Context-free language
Type 3	Regular grammar	Regular language

Take a look at the following illustration. It shows the scope of each type of grammar.



Type-0 grammars generate recursively enumerable languages. The productions have no restrictions. They are any phase structure grammar including all formal grammars.

The productions can be in the form of **α → β** where **α** is a string of terminals and non-terminals with at least one non-terminal and **α** cannot be null. **β** is a string of terminals and non-terminals.

Example:

S → ACaB
Bc → acB
CB → DB
aD → Db
BCDEF->a

Type-1 grammars generate context-sensitive languages. The productions must be in the form

α A β → α γ β

where **A ∈ N** (Non-terminal)

and **α, β, γ ∈ (T ∪ N)*** (Strings of terminals and non-terminals)

The strings **α** and **β** may be empty, but **γ** must be non-empty.

The rule **S → ε** is allowed if S does not appear on the right side of any rule. The languages generated by these grammars are recognized by a linear bounded automaton.

Example:

AB → AbBc
A → bcA
B → b

Type-2 grammars generate context-free languages.

The productions must be in the form **A → γ**

where **A ∈ N** (Non terminal)

and **γ ∈ (T ∪ N)*** (String of terminals and non-terminals).

Example:

S → X a
X → a
X → aX
X → abc
X → ε

Type-3 grammars generate regular languages. Type-3 grammars must have a single non-terminal on the left-hand side and a right-hand side consisting of a single terminal or single terminal followed by a single non-terminal.

The productions must be in the form **X → a or X → aY**

where **X, Y ∈ N** (Non-terminal)

and **a ∈ T** (Terminal)

The rule **S → ε** is allowed if **S** does not appear on the right side of any rule.

Example:

X → ε
X → a | aY
Y → b

Decidability

A language $L \subseteq \Sigma^*$ is **decidable** if and only if (iff) the characteristic function χ of L, i.e. χ_L: $\Sigma^* \rightarrow \{0, 1\}$ with

$$\chi_L(x) = \begin{cases} 1, & w \in L \\ 0, & w \notin L \end{cases} \qquad Def(\chi_L) = \Sigma^*$$

is computable.

A language $L \subseteq \Sigma^*$ is **semi-decidable** iff the semi-characteristic function χ^l of L, *i.e.* χ_L^l: $\Sigma^* \rightarrow \{0, 1\}$ with

$$\chi_L^l(x) = \begin{cases} 1, & w \in L \\ \perp, & w \notin L \end{cases} \qquad Def(\chi_L^l) = L$$

is computable.

Relationship between decidable and semi-decidable languages

A language $L \subseteq \Sigma^*$ is **decidable** iff L and $\bar{L} = \Sigma^* - L$ are semi-decidable.

$$\chi_L^l(x) = \begin{cases} 1, & \chi_L(x) = 1 \\ \perp, & \chi_L(x) = 0 \end{cases}$$

$$\chi_{\bar{L}}^l(x) = \begin{cases} 1, & \chi_{\bar{L}}(x) = 0 \\ \perp, & \chi_{\bar{L}}(x) = 1 \end{cases}$$

$=>$ χ_L^l and χ_L^L are computable. The two algorithms should be cleverly brought together.

Decidability in the context of word problem:

Decidability of the word problem for a language class

te_Σ: C_Σ x Σ^* → {0, 1}: $te_\Sigma(L, w) = \begin{cases} 1, & w \in L \\ 0, & w \notin L \end{cases}$

Here: C_Σ = Typ-i_Σ, $0 \leq i \leq 3$

Consider G = (Σ, N, P, S), w ∈ Σ^*
➔ Question: w ∈ L(G)?

Typ 3: A → aB | ε

Typ 2: A → BC | a

Typ 1: $(\Sigma \cup N)^* - \Sigma^* \times (\Sigma \cup N)^*$ with $|\alpha| \leq |\beta|$ **(=> Monotony property)**

Typ 0: ?

Computability

How to formally specify an algorithm?

➔ Turing (Thue, Post, Klune, Church, …)

Turing Machine

A Turing Machine (TM), thought of by the mathematician Alan Turing in 1936, is a mathematical model which consists of an infinite length tape divided into cells on which input is given. It consists of a head which reads the input tape. A state register stores the state of the Turing machine. After reading an input symbol, it is replaced with another symbol, its internal state is changed, and it moves from one cell to the right or left. If the TM reaches the final state, the input string is accepted, otherwise rejected. Despite its simplicity, the TM can simulate ANY computer algorithm, no matter how complicated it is!

A TM can be formally described as a 7-tuple (Σ, T, S, δ, s_0, #, F) where

- **Σ** is the input alphabet
- **T** is the tape alphabet (wheras **Σ ⊂ T**)
- **S** is a finite set of states
- **δ** is a transition function; δ : S × T → S × T × {left_shift, right_shift, or no_movement}.
- **s_0** is the initial state
- **#** is the blank symbol
- **F** is the set of final states

TM Example 1: Bit Inversion

With the symbols "1 1 0" printed on the tape, let's attempt to convert the 1s to 0s and vice versa. This is called bit inversion, since 1s and 0s are bits in binary. This can be done by passing the following instructions to the Turing machine, utilizing the machine's reading capabilities to decide its subsequent operations on its own. These instructions make up a simple program.

Turing machine M = (Σ, T, S, δ, S_0, #, F) with

- Σ = {1, 0}
- T = {1, 0, #}
- S = {s_0, s_f }
- S_0 = {s_0}
- # = blank symbol
- F = {s_f }

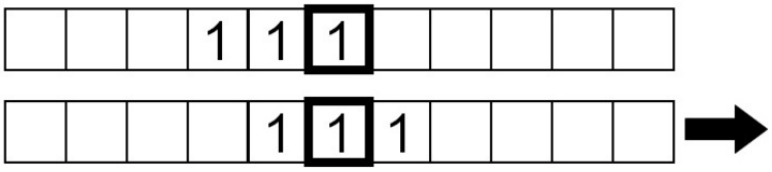
δ is given by

Current state	Symbol read		Symbol written	New state	Move instruction
s_0	Blank (#)	→	None	s_f	None
s_0	0	→	Write 1	s_0	Move tape to the right
s_0	1	→	Write 0	s_0	Move tape to the right

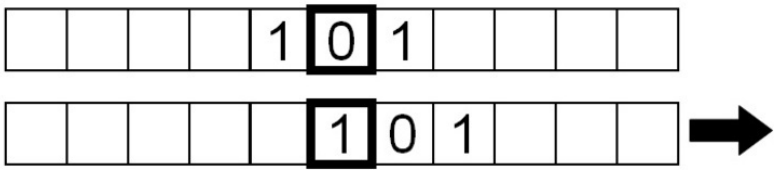
The machine will first read the symbol under the head, write a new symbol accordingly, then move the tape left or right as instructed, before repeating the read-write-move sequence again.

Let's see what this program does to our tape from the previous end point of the instructions:

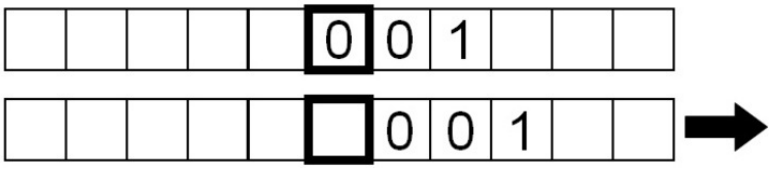
The current symbol under the head is 0, so we write a 1 and move the tape right by one square.



The symbol being read is now 1, so we write a 0 and move the tape right by one square:



Similarly, the symbol read is a 1, so we repeat the same instructions.



Finally, a 'blank' symbol is read, so the machine does nothing apart from read the blank symbol continuously since we have instructed it to repeat the read-write-move sequence without stopping.

After every instruction, we also specify a state for the machine to transition to. In the example, the machine is redirected back to its original state, State 0, to repeat the read-write-move sequence, unless a blank symbol is read. When the machine reads a blank symbol, the machine is directed to a stop state and the program terminates.

TM Example 2: Doubling the Number of Ones

The following deterministic single-tape Turing machine M expects a sequence of ones as input on the tape. It doubles the number of ones, with a blank symbol in the middle. For example, "11" becomes "11011". The read / write head is initially on the first one. The initial state is s_0, the end state is s_5. The zero stands for the empty field (i.e., 0 instead of #) and the tape consists of empty fields except for the ones written on it.

Turing machine $M = (\Sigma, T, S, \delta, S_0, \#, F)$ with

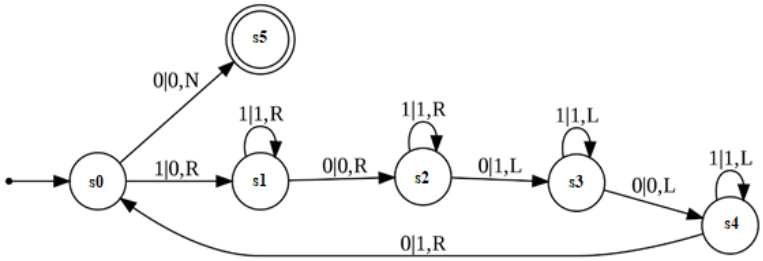
- $\Sigma = \{1\}$
- $T = \{1, 0\}$
- $S = \{s_0, s_1, s_2, s_3, s_4, s_5\}$
- $S_0 = \{s_0\}$
- # = blank symbol as 0

- $F = \{S_5\}$

δ is given by

Current state	Symbol read		Symbol written	New state	Move instruction
s_0	1	→	0	s_1	Right
s_0	0	→	0	s_5	None
s_1	1	→	1	s_1	Right
s_1	0	→	0	s_2	Right
s_2	1	→	1	s_2	Right
s_2	0	→	1	s_3	Left
s_3	1	→	1	s_3	Left
s_3	0	→	0	s_4	Left
s_4	1	→	1	s_4	Left
s_4	0	→	1	s_0	Right

or



In the above example, when entering "11", M runs through the following states, with the current head position being printed in bold:

Stage	State	Tape
1	s_0	**1**1000
2	s_1	0**1**000
3	s_1	01**0**00
4	s_2	010**0**0
5	s_3	010**1**0
6	s_4	0**1**010
7	s_4	**0**1010
8	s_0	1**1**010

Stage	State	Tape
9	s_1	10**0**10
10	s_2	100**1**0
11	s_2	1001**0**
12	s_3	1001**1**
13	s_3	10**0**11
14	s_4	**1**0011
15	s_0	11**0**11
16	s_5	-stop-

Pushdown automata (PDA) with 2 stacks (2PDA)
$2PDA_{\Sigma} = TA_{\Sigma}$

Q_{Σ}: Queue Automat (First in First out – FIFO)
$Q_{\Sigma} = TA_{\Sigma}$

In addition to Turing computability, two further computability terms are the Queue computability and the 2PDA computability. Note that they are many other computability concepts available, including Universal Register Machines (URM), λ calculus, and Markov.

Computability with programming language approaches: **Loop, While, Goto**

loop x **do** A **end**;

while x ≠0 **do** A **end**;

label x; A **goto** x;

Relationship:

⇨ loop ⊂ while = goto = Turing computability

It is known that

⇨ f: loop-computable functions → total computable functions

Question: How about the other direction?

⇨ loop-computable functions ← total computable functions **?**

∉

Ackermann function

Church's thesis: The class of functions that can be calculated in the intuitive sense is precisely the class of functions that can be calculated by Turing-computable functions.